

MifosX4MM — Week 1: Learning & Exploring (Jun 1–5)

📖 Hub: [MifosX4MM — Master Plan Hub](#) | 🔍 Reference: [2026-05-31-mifos-critical-analysis](#)

Pure learning this week. No code fixes. No mobile. No KBZ Pay.

Scope

✅ In Scope	❌ Out of Scope
Domain knowledge (microfinance, PAR, loan lifecycle)	apps/mobile — React Native app
Docker infrastructure + running all 7 services	services/mobile-money — KBZ Pay
packages/shared-types — the data contract	Any code fixes or new features
Keycloak auth + JWT verification flow	Production deployment
API Gateway — all 5 route files	KBZ Pay UAT credentials
Web portal — Next.js, React Query	
KYC service — pluggable provider pattern	
Reporting service — Python + direct SQL	
Database schema — MySQL + PostgreSQL	
PAR SQL calculation	

Monday — Domain Knowledge + First Boot

Theme: Understand what this system IS before looking at any code.

Hours: ~3 hrs | Everyone works together

Morning: Domain Theory (1 hr)

☐ Read README.md completely

File: D:\Mifos\MifosX4MM\README.md

Don't skim. Read every section including the architecture diagram.

☐ **Read ARCHITECTURE.md — focus on Design Principles**

File: D:\Mifos\MifosX4MM\docs\ARCHITECTURE.md

Read the 4 Design Principles at the top. These explain every decision in the codebase.

☐ **Write answers to these questions** (in your own words, in a note):

1. What is a microfinance institution (MFI)? What does it do differently from a bank?
2. What is PAR (Portfolio at Risk)? What does PAR30 mean specifically?
3. What is the difference between loan *disbursement* and loan *repayment*?
4. Why is Apache Fineract used instead of building a custom banking system from scratch?
5. What are the 5 user roles? What can a `loan_officer` do that a `teller` cannot?

Afternoon: Install + Run the Full Stack (2 hrs)

Person B drives. Person A watches and follows along on their own machine.

☐ **Install prerequisites** (if not already done):

- Docker Desktop: <https://www.docker.com/products/docker-desktop/>
- Node.js 20+: <https://nodejs.org/>
- pnpm: `npm install -g pnpm`
- Python 3.12+: <https://python.org/>

☐ **Clone and install dependencies:**

```
cd D:\Mifos\MifosX4MM
pnpm install
cp .env.example .env
```

☐ **Start all Docker containers:**

```
pnpm docker:up
```

 Wait **3–5 minutes**. Do not proceed until Fineract finishes its database migrations. Watch for this in the logs: `Started Fineract API application`

☐ **Start development services:**

```
pnpm dev
```

☐ **Verify every service is alive:**

```
curl http://localhost:3001/health # → {"status":"ok","service":"api"}
curl http://localhost:3003/health # → {"status":"ok","service":"mobile-money"}
curl http://localhost:3004/health # → {"status":"ok","service":"kyc"}
curl http://localhost:3005/health # → {"status":"ok","service":"reporting"}
```

Also check:

- `http://localhost:3000` → web portal loads
- `http://localhost:8080/fineract-provider/actuator/health` → Fineract responds

☐ **Login to web portal:**

- URL: `http://localhost:3000`
- Username: `admin` | Password: `Admin@1234`
- You'll be forced to change the password — write it down

☐ **Seed test data:**

```
pnpm seed
```

☐ **Explore the web portal** — click every single menu item:

- Dashboard → note the 4 KPI cards (active clients, active loans, PAR30, collections today)
- Clients list → open one client
- Loans list → open one loan → see the repayment schedule
- Collections
- Reports
- Settings

☐ **Login as** `loan.officer` (open a new browser tab or incognito):

- Password: `Officer@1234` (change when prompted)
- Notice what menu items are missing compared to admin
- This is RBAC in action — written in one line: `app.authorize(['branch_manager', 'super_admin'])`

✅ **Monday done when:** Every team member has the system running. Dashboard shows real numbers. You can answer all 5 domain questions above without looking.

Tuesday — Auth System + Shared Types

Theme: Understand the data contract and how authentication works end-to-end.

Hours: ~3 hrs | Person B leads auth, everyone reads shared types

Shared Types: The Contract Between All Services (1 hr)

- ☐ Read the entire shared types file:

```
File: D:\Mifos\MifosX4MM\packages\shared-types\src\index.ts
```

This file is ~209 lines. Read every interface. For each one, write one sentence:

Interface	What it represents
FineractClient	
FineractLoanAccount	
FineractLoanRepayment	
KycSubmission	
KycVerificationRequest	
DashboardStats	
AuthUser	
UserRole	

- ☐ Understand KycStatus lifecycle:

```
pending → processing → approved  
                → rejected  
                → manual_review
```

A loan officer submits a KYC request → it goes pending → the provider processes it → result is approved or rejected.

- ☐ Understand DashboardStats fields:

- par0 = any loan with even 1 day overdue
- par30 = loans with 30+ days overdue (the headline figure)
- par90 = loans with 90+ days overdue (severe)
- collectionsToday = total MMK collected today

Auth System: How JWT Works Here (2 hrs)

- ☒ Read the auth code:

- ☐ Read the Keycloak plugin:

```
File: D:\Mifos\MifosX4MM\apps\api\src\plugins\keycloak.ts
```

Trace this flow in your head:

1. Request arrives at a protected route
2. `app.authenticate` hook fires → calls `req.jwtVerify()`
3. JWT header contains `kid` (key ID)
4. `jwt-rsa` fetches the matching public key from Keycloak (cached 10 min)
5. JWT is verified: RS256 signature + issuer + expiry checked
6. `formatUser` maps raw Keycloak claims → `AuthUser { id, username, email, roles }`
7. `req.user` is now available in the route handler

☐ **Read the auth routes:**

File: `D:\Mifos\MifosX4MM\apps\api\src\routes\auth.ts`

☐ **Read the Keycloak realm config:**

File: `D:\Mifos\MifosX4MM\infra\keycloak\realm-mifos.json`

Find the `clients` array. Find `mifos-staff` — this is what the web portal uses to login.
Find the `users` array — see the default test users.

 Trace it live in the browser:

- ☐ Open `http://localhost:3000` → Open DevTools (F12) → Network tab → Login
- ☐ Find the `POST /api/v1/auth/login` request
- ☐ Look at the response — find the `accessToken` field
- ☐ Copy the token → paste into <https://jwt.io>
- ☐ In the Payload section, find:
 - `preferred_username` → your username
 - `realm_access.roles` → the list of roles assigned to you
 - `exp` → expiry timestamp (convert at <https://epochconverter.com> — should be ~15 min from login)
- ☐ Open Keycloak admin: `http://localhost:8180/admin` (`admin / admin`)
 - Go to: Realm `mifos` → Users — see `admin` and `loan.officer`
 - Go to: Realm `mifos` → Roles — see all 5 role definitions
 - Go to: Realm `mifos` → Clients → `mifos-staff` → Settings
 - Note: `Direct Access Grants Enabled`: `ON` — this is what allows username/password login
- ☐ **Create a test user manually** to confirm you understand Keycloak:
 - Realm `mifos` → Users → Add user
 - Username: `explore.teller` | Email: anything
 - Credentials tab → Set Password: `Teller@1234` | Temporary: `OFF`
 - Role Mappings tab → Assign role: `teller`

- Login at `http://localhost:3000` — verify `teller` sees even fewer options than `loan.officer`

✅ **Tuesday done when:** You can explain the JWT auth flow without looking at the code. You can decode a JWT and find the roles in the payload.

Wednesday — API Gateway + Fineract

Theme: Understand the brain of the system — every route, every Fineract call.

Hours: ~3.5 hrs | Person B leads, Person A follows along

API Gateway: Read Every File (2 hrs)

☐ **Read the entry point:**

```
File: D:\Mifos\MifosX4MM\apps\api\src\index.ts
```

Note the startup order:

1. Create Fastify
2. Register CORS
3. Register Keycloak plugin (adds `app.authenticate` and `app.authorize`)
4. Create Fineract client
5. Register public routes (auth — no JWT needed)
6. Register protected routes (JWT required for all)
7. Set global error handler

☐ **Read the Fineract client:**

```
File: D:\Mifos\MifosX4MM\apps\api\src\fineract.ts
```

Notice:

- Uses HTTP Basic Auth (username + password) — this is Fineract's standard auth method
- Always sends `Fineract-Platform-TenantId` header — Fineract is multi-tenant
- Error interceptor: converts Fineract's verbose Java errors into clean error objects

☐ **Read ALL 5 route files. For each one, list what it does:**

```
File: D:\Mifos\MifosX4MM\apps\api\src\routes\auth.ts
```

→ login, refresh, logout, /me (get current user)

```
File: D:\Mifos\MifosX4MM\apps\api\src\routes\clients.ts
```

→ list clients, get client, create client, update client

```
File: D:\Mifos\MifosX4MM\apps\api\src\routes\loans.ts
```

→ list loans, loans by client, get loan detail, post repayment, approve/disburse/reject

⚠ **Mark line 15** — you'll see the SQL injection bug here

```
File: D:\Mifos\MifosX4MM\apps\api\src\routes\dashboard.ts
```

→ calls the reporting service to aggregate stats

```
File: D:\Mifos\MifosX4MM\apps\api\src\routes\payments.ts
```

→ initiates KBZ Pay orders (skip this one — out of scope this week)

Fineract: Explore the Core Banking Engine (1.5 hrs)

☐ Open Fineract Swagger UI:

```
http://localhost:8080/fineract-provider/swagger-ui/index.html
```

Browse the available endpoints. Notice:

- `/clients` — borrower management
- `/loans` — loan account management
- `/loans/{loanId}` — loan detail
- `/loans/{loanId}/transactions` — post repayments, disbursements
- `/loanproducts` — loan product configuration

☐ Call the Fineract API directly (bypassing the gateway):

```
# List all loan products:
curl -u "mifos:password" \
  -H "Fineract-Platform-TenantId: default" \
  http://localhost:8080/fineract-provider/api/v1/loanproducts
```

```
# List active clients:
curl -u "mifos:password" \
  -H "Fineract-Platform-TenantId: default" \
  "http://localhost:8080/fineract-provider/api/v1/clients?
offset=0&limit=10"
```

Compare this raw Fineract response with what `GET /api/v1/clients` returns from your gateway. Notice how the gateway cleans up and shapes the data.

☐ **Trace a loan fetch end-to-end:**

1. Open `http://localhost:3000/loans` in the browser
2. DevTools → Network → find `GET /api/v1/loans`
3. Open `apps/api/src/routes/loans.ts` → find that handler
4. See the call to `fineract.get('/loans', { params })`
5. Open `apps/api/src/fineract.ts` → see the Axios instance that sends this to Fineract
6. That Axios call hits `http://fineract:8080/fineract-provider/api/v1/loans` inside Docker

Draw this on paper:

```
Browser → GET /api/v1/loans
  → Fastify gateway (apps/api) → loans.ts handler
  → fineract.get('/loans') → Axios instance (fineract.ts)
  → HTTP Basic Auth → Fineract :8080
  → MySQL database (fineract_default)
  → response shaped → returned to browser
```

☒ **Wednesday done when:** You can trace any web portal action to its Fineract API call and back. You understand what the gateway adds on top of raw Fineract.

Thursday — KYC Service + Reporting + Database

Theme: Understand the two supporting services and the data that powers the dashboard.

Hours: ~3.5 hrs | Person B leads reporting (Python), Person A leads KYC

KYC Service: The Pluggable Pattern (1.5 hrs)

☐ **Read the provider interface:**

```
File: D:\Mifos\MifosX4MM\services\kyc\src\provider.ts
```

This interface has exactly 4 methods:

- `submit()` — start a verification → returns `KycSubmission` with `status: 'processing'`
- `getStatus()` — check current status
- `handleWebhook()` — receive async callback from provider

- `getStats()` — counts by status

☐ **Read the stub provider:**

```
File: D:\Mifos\MifosX4MM\services\kyc\src\providers\stub.ts
```

See: `private submissions = new Map<string, KycSubmission>()`

This is an in-memory JavaScript Map. When the container restarts, this Map is gone.

This is Bug #4.

☐ **Read the service entry point:**

```
File: D:\Mifos\MifosX4MM\services\kyc\src\index.ts
```

Find `resolveProvider()` — this is where `KYC_PROVIDER=stub` env var selects the stub.

☐ **Read the routes:**

```
File: D:\Mifos\MifosX4MM\services\kyc\src\routes.ts
```

Notice: routes call `provider.submit()`, `provider.getStatus()` — they don't know or care which provider is active. This is the pluggable pattern.

☐ **Test the KYC service manually:**

```
# Submit a verification:
curl -X POST http://localhost:3004/kyc/submit \
  -H "Content-Type: application/json" \
  -d '{
    "clientRef": "test-client-001",
    "documentType": "nrc",
    "firstName": "Mg",
    "lastName": "Aung",
    "dateOfBirth": "1990-01-01",
    "documentNumber": "12/AYK(N)000001",
    "frontImageBase64": "dGVzdA=="
  }'

# Note the submissionId in the response

# Wait 3 seconds, then check status (stub auto-approves after 2
seconds):
curl http://localhost:3004/kyc/status/YOUR_SUBMISSION_ID
# Status should be: "approved"

# Now prove Bug #4 – restart KYC and check again:
docker restart mifos_kyc
curl http://localhost:3004/kyc/status/YOUR_SUBMISSION_ID
# Status: 404 – data is gone
```

Reporting Service: Python + Direct SQL (2 hrs)

☐ Read the main entry point:

```
File: D:\Mifos\MifosX4MM\services\reporting\main.py
```

Three routers: `portfolio`, `collections`, `kyc_summary`

Startup: verifies PostgreSQL connectivity (`SELECT 1`) — fails fast if DB is unreachable

☐ Read the database connection:

```
File: D:\Mifos\MifosX4MM\services\reporting\db.py
```

Uses SQLAlchemy async with `asyncpg` driver → connects to PostgreSQL
`fineract_default`

☐ Read the portfolio router:

```
File: D:\Mifos\MifosX4MM\services\reporting\routers\portfolio.py
```

This is where PAR is calculated. The SQL uses a CTE on `m_loan_repayment_schedule`.

☐ Read the collections router:

```
File: D:\Mifos\MifosX4MM\services\reporting\routers\collections.py
```

Calculates today's repayments: `SUM(amount) FROM m_loan_transaction WHERE transaction_date = CURRENT_DATE`

☐ Prove Bug #3 — no auth on reporting:

```
# No token - still returns sensitive financial data:
curl http://localhost:3005/reports/portfolio/summary
# Returns: PAR figures, outstanding amounts, overdue totals
# This is a security hole
```

☐ Connect directly to PostgreSQL and run queries:

```
docker exec -it mifos_postgres psql -U postgres -d fineract_default
```

Once inside psql:

```
-- See all Fineract tables:
\dt m_*

-- Count active loans (status 300 = active):
SELECT COUNT(*) FROM m_loan WHERE loan_status_id = 300;

-- Count active clients:
SELECT COUNT(*) FROM m_client WHERE status_enum = 300;
```

```
-- See the most recent transactions:
SELECT loan_id, transaction_type_enum, transaction_date, amount
FROM m_loan_transaction
ORDER BY created_date DESC
LIMIT 10;
```

☐ **Run the PAR30 calculation yourself:**

```
-- Find loans with overdue installments:
WITH overdue AS (
  SELECT
    loan_id,
    MIN(duedate) AS oldest_overdue_date,
    CURRENT_DATE - MIN(duedate) AS days_in_arrears
  FROM m_loan_repayment_schedule
  WHERE duedate < CURRENT_DATE
    AND completed_derived = false
    AND obligations_met_on_date IS NULL
  GROUP BY loan_id
)
SELECT
  loan_id,
  days_in_arrears,
  CASE
    WHEN days_in_arrears >= 90 THEN 'PAR90'
    WHEN days_in_arrears >= 30 THEN 'PAR30'
    ELSE 'PAR0'
  END AS par_bucket
FROM overdue
ORDER BY days_in_arrears DESC;
```

Compare your result with the PAR30 number on the dashboard. Do they match?

☐ **Exit psql:**

```
\q
```

✓ **Thursday done when:** You can explain what the KYC provider pattern is and why it matters. You can run the PAR query and explain what it means. You know exactly why Bug #3 and Bug #4 are dangerous.

Friday — Review + Show Outcomes

Theme: Prove your understanding. Answer the mastery questions. Show what you can do.

Hours: ~2.5 hrs

Morning: Mastery Questions (1 hr)

Answer these **without looking at the code**. If you can answer all 10, Week 1 is complete.

- ☐ 1. What is Apache Fineract and why is it used instead of building a custom banking system?
 - ☐ 2. Why are there TWO databases (MySQL and PostgreSQL)? Which service owns each one?
 - ☐ 3. Why does the reporting service query SQL directly instead of calling Fineract's REST API?
 - ☐ 4. How does JWT verification work without calling Keycloak on every request?
 - ☐ 5. What are the 4 methods of the `KycProvider` interface and what does each one do?
 - ☐ 6. What does `loan_status_id = 300` mean in the `m_loan` table?
 - ☐ 7. What happens if a KYC submission is made and then the KYC container restarts?
 - ☐ 8. What does `days_in_arrears >= 30` mean in the PAR calculation?
 - ☐ 9. What is the `Fineract-Platform-TenantId` header and why is it sent on every request?
 - ☐ 10. What would break if `packages/shared-types` changed the `FineractLoanRepayment` interface?
-

Show These 6 Outcomes (1.5 hrs)

Outcome 1 — Full Stack Running (5 min)

Show all 7 services healthy:

```
curl http://localhost:3001/health
curl http://localhost:3003/health
curl http://localhost:3004/health
curl http://localhost:3005/health
docker ps # all containers running
```

Open `http://localhost:3000` — show the dashboard with real data.

Outcome 2 — RBAC Live Demo (3 min)

Open two browser tabs side by side:

- Tab 1: Logged in as `admin`

- Tab 2: Logged in as `loan.officer`

Show: Approve Loan button exists for admin, not for loan.officer.

Explain: "The role check is in `apps/api/src/routes/loans.ts` — `app.authorize(['branch_manager', 'super_admin'])`. The frontend doesn't control this."

Outcome 3 — JWT Decoded Live (3 min)

1. Login as admin in the browser
2. DevTools → Network → find the login response
3. Copy the `accessToken`
4. Go to <https://jwt.io> → paste it
5. Show `realm_access.roles` in the payload

Explain: "The token is signed with RS256 by Keycloak. The gateway verifies it using JWKS — Keycloak's public key — cached for 10 minutes. No database call on every request."

Outcome 4 — Fineract Swagger Live (3 min)

Open <http://localhost:8080/fineract-provider/swagger-ui/index.html>

Show:

- The `POST /loans/{loanId}/transactions` endpoint
 - Explain: "This is what the gateway calls when a repayment is confirmed. The gateway is just a secure, typed wrapper around Fineract's REST API."
-

Outcome 5 — PAR Query Running Live (5 min)

```
docker exec -it mifos_postgres psql -U postgres -d fineract_default
```

Run the PAR CTE query from Thursday. Show the results.

Explain: "The reporting service runs this query directly on PostgreSQL — not through Fineract's REST API. This is more accurate because Fineract's `inArrears` flag only updates after a scheduled batch job."

Outcome 6 — Bug Report (5 min)

Open [2026-05-31-mifos-critical-analysis](#) in Obsidian.

Walk through the 5 bugs table. For each one:

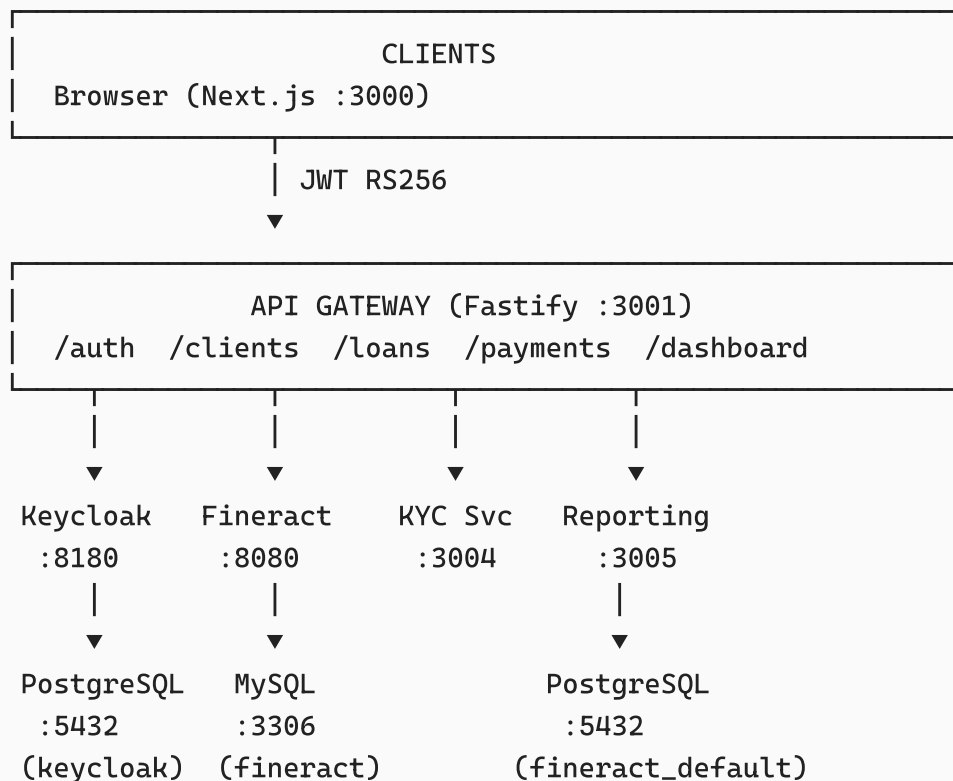
- Point to the exact file and line
- Explain in one sentence why it's dangerous
- State which week it will be fixed

Show Bug #3 live:

```
curl http://localhost:3005/reports/portfolio/summary
# No token. Returns PAR data, outstanding amounts. This is the bug.
```

Architecture Diagram (draw it yourself)

During the week, draw this on paper or in Obsidian Canvas:



This diagram, drawn in your own hand, proves you understand the system.

Week 1 Completion Checklist

- ☐ All 7 services running locally (all health endpoints return ok)
 - ☐ Seeded with test data (dashboard shows real KPI numbers)
 - ☐ Can explain all 5 domain concepts (PAR, disbursement, repayment, loan officer role, MFI)
 - ☐ JWT decoded live in jwt.io — roles visible in payload
 - ☐ Keycloak admin explored — users, roles, clients all understood
 - ☐ All 5 API gateway route files read and understood
 - ☐ Fineract Swagger explored — at least 5 endpoints examined
 - ☐ A loan fetch traced end-to-end: browser → gateway → Fineract → MySQL
 - ☐ KYC pluggable pattern understood — stub provider read fully
 - ☐ Bug #4 (KYC data loss) demonstrated live with docker restart
 - ☐ Reporting service Python code read and understood
 - ☐ PAR SQL query run live in psql — result matches dashboard
 - ☐ Bug #3 (reporting no auth) demonstrated live with curl
 - ☐ Architecture diagram drawn (hand or Obsidian Canvas)
 - ☐ All 10 mastery questions answered without looking at code
-

What You Will NOT Have on Friday

Being honest about what one week of learning produces:

- **✗** No code fixes committed yet (that's Week 2–3)
- **✗** No KBZ Pay integration (that's Week 5–6)
- **✗** No production deployment
- **✗** No new features

And that's correct. A junior team that deeply understands a complex financial system after one week of focused study is exactly where you should be. Week 2 builds on this foundation. Every fix in Week 3 will be done with full confidence — no guessing.

Week 1 Learning Plan · Created: 2026-06-01 · Focus: Understanding only · Project: MifosX4MM